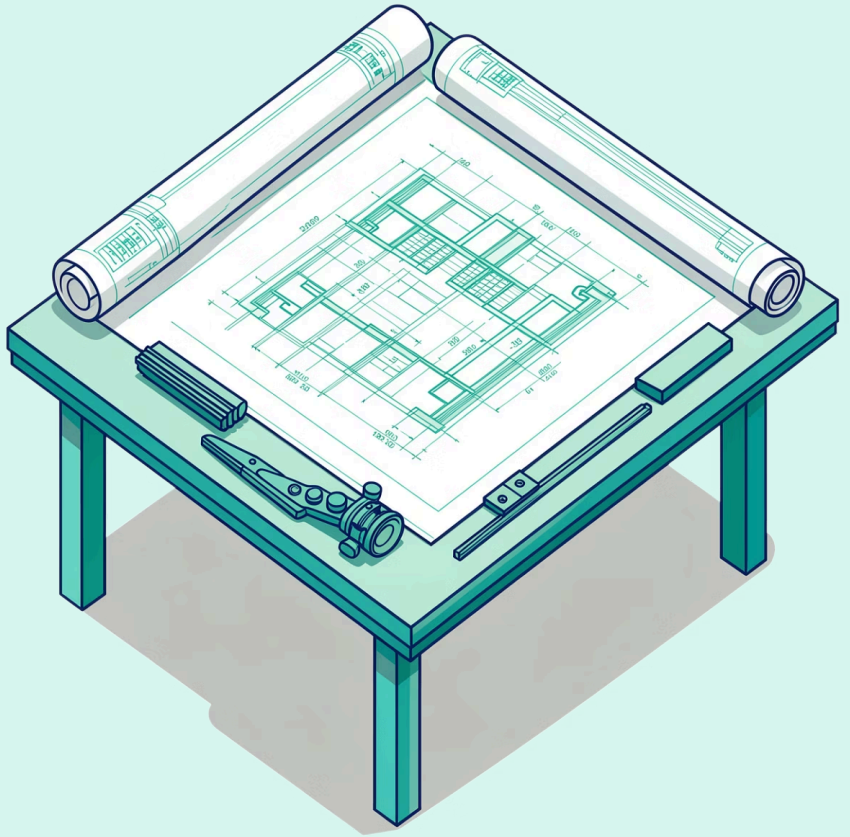


CEN 4065C: Modeling with UML

Functional, Structural & Behavioral Views

Course

CEN 4065C – Software Architecture & Design

- **Core Topic:** Unified Modeling Language (UML) views and notations
- **Primary Text:** Bruegge & Dutoit, *Object-Oriented Software Engineering*, Chapter 2
- **Objective:** Master functional, structural, and behavioral system modeling

Made with GAMMA

Lecture Agenda & Roadmap

01

Part 1

Motivation, Prerequisites & Misconceptions *(Slides 1–8)*

02

Part 2

Fundamental Modeling Concepts & Terminology *(Slides 9–18)*

03

Part 3

Core UML Views: Functional, Structural & Behavioral *(Slides 19–30)*

04

Part 4

FRIEND Case Study & PlantUML Execution *(Slides 31–38)*

05

Part 5

Tradeoffs, Best Practices, Assessments & Homework *(Slides 39–50)*

Made with GAMMA

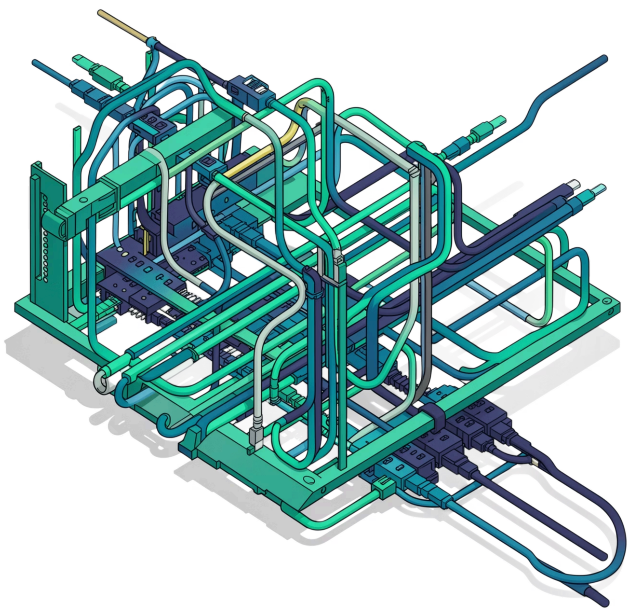
Course Learning Outcomes & Syllabus Competencies

1	**Outcome 1** Describe core software framework activity principles
2	**Outcome 2** Elicit domain models and list conceptual classes
3	**Outcome 3** Construct detailed static class diagrams
4	**Outcome 4** Document dynamic behavior using interaction and activity diagrams

Aligned with Form 112 Competency 2 and 3 requirements



Made with GAMMA



Architectural Motivation: Managing System Complexity

System Scale

Modern software quickly exceeds individual cognitive capacity

Pirsig's Principle

Parts are hard to locate when treated as sub-components of something else

Core Strategy

Apply divide-and-conquer through focused system abstractions

Role of Notations

Enable precise, unambiguous team communication

Made with GAMMA

Prerequisite Review: Object-Oriented Fundamentals

COP 3530 – DATA STRUCTURES

Classes vs. Objects

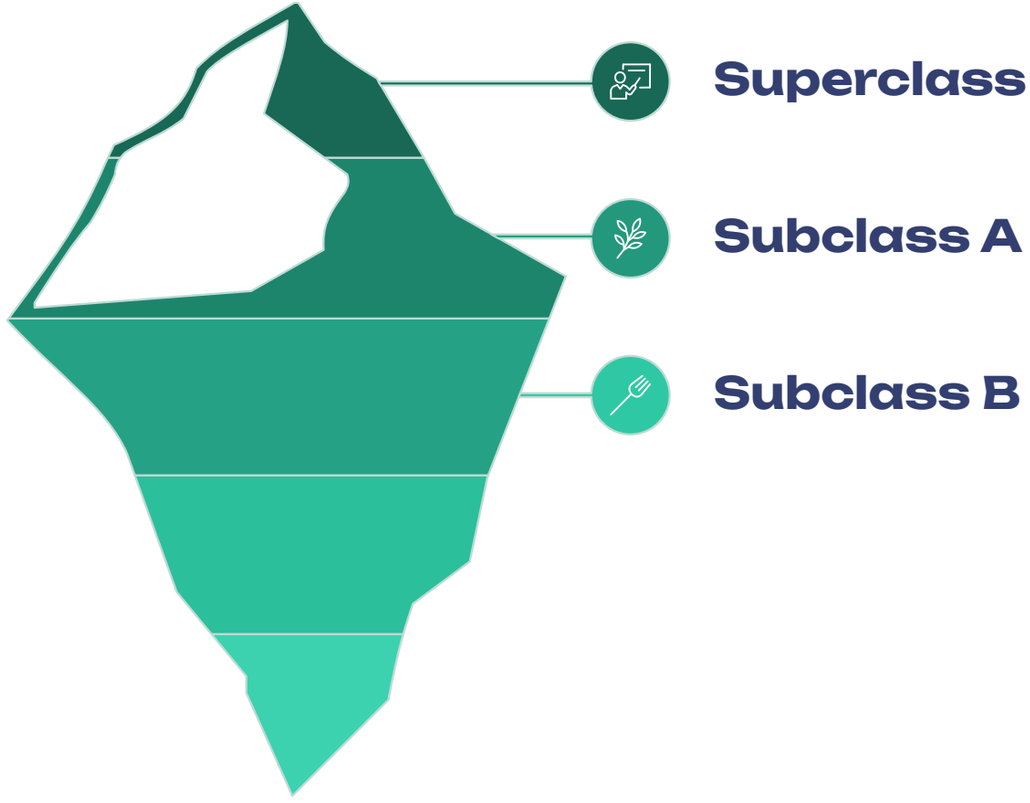
Classes define templates; objects are runtime instances

State & Behavior

Attributes store values; operations define callable logic

Inheritance

Specialization relationship between superclasses and subclasses



Made with GAMMA

Prerequisite Review: System Execution & Runtime Memory

Runtime Memory

Objects exist as concrete instances at specific memory addresses

Execution Streams

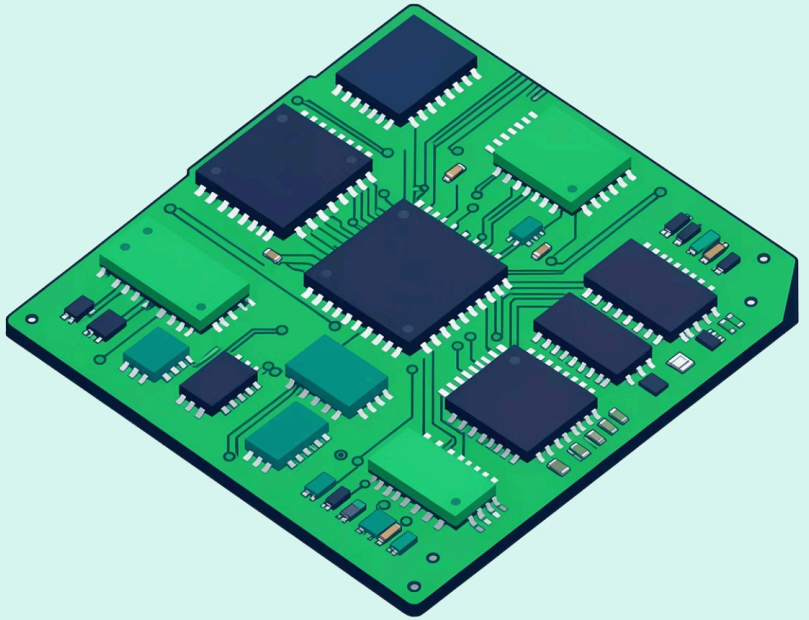
Sequential method calls vs. asynchronous event messages

Associations as Pointers

Static class relationships translate to object references or data structures

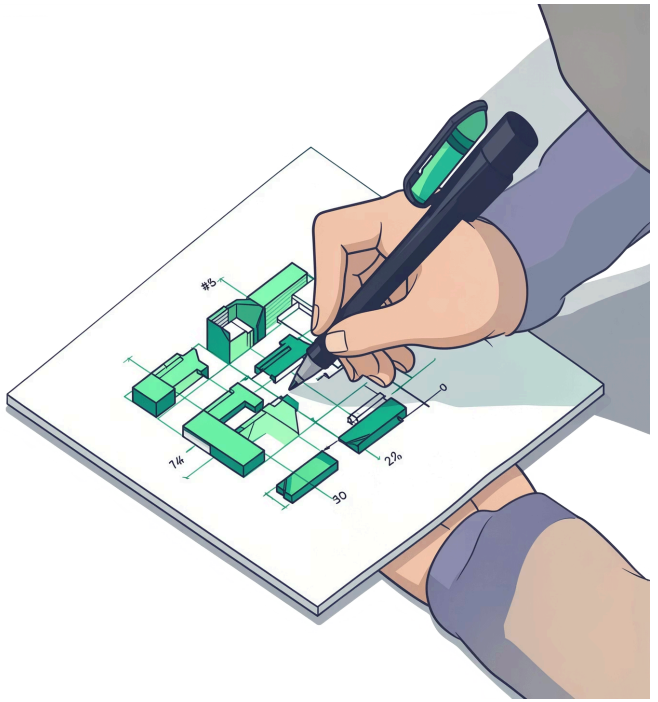
System Boundary

Encloses software objects, separating them from external entities



Made with GAMMA

Common Misconceptions: UML as Blueprints vs. Agile Sketches



❌ Misconception 1

"UML is strictly for exhaustive documentation prior to coding"

✔ Reality

UML serves as an **agile sketch tool** for analysis and design communication

❌ Misconception 2

"A single diagram type can capture an entire system"

✔ Reality

Systems require **multiple overlapping views** — functional, structural, and dynamic

Made with GAMMA

Common Misconceptions: Conflating Application & Solution Domains

Application Domain

Real-world problem space and user environment

- Aircraft
- TrafficController

Elicit these concepts **first** — before introducing any solution objects

Solution Domain

Architecture space of executable software systems

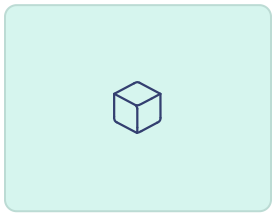
- MapDisplay
- FlightPlanDatabase

Introduced only after application domain is fully understood

⚠ Common Pitfall: Prematurely inserting database schemas or UI buttons into requirements models violates the architectural rule — elicit application domain concepts first.

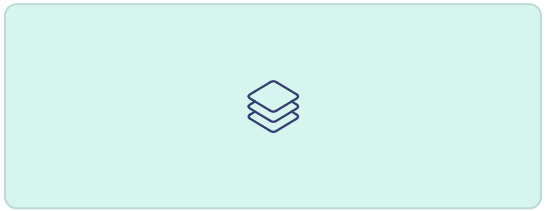
Made with GAMMA

Fundamental Definitions: System, Subsystem & Object Recursion



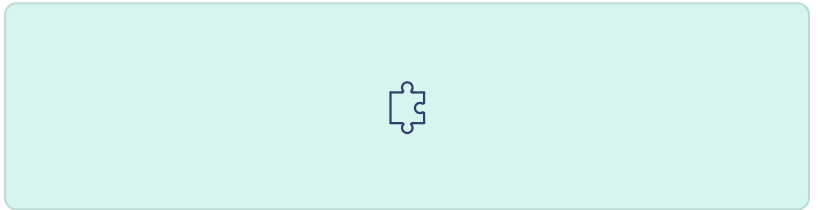
Object Boundary

End of recursion — components simple enough to comprehend without further breakdown



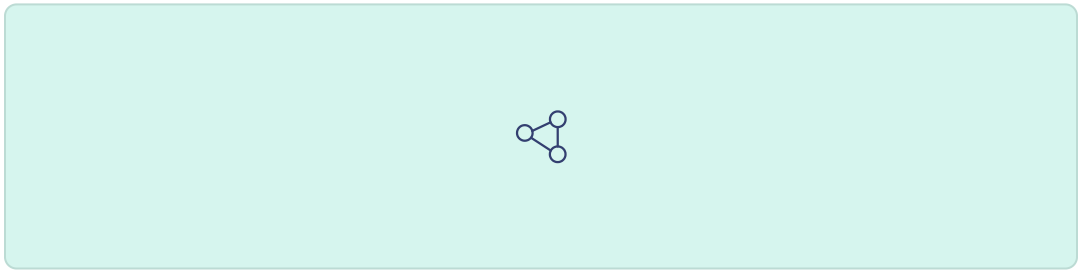
Recursive Decomposition

Iterative breakdown of subsystems to manage architectural scale



Subsystem

System component that is itself a simpler organized system



Engineered System

Organized set of communicating parts designed for a specific purpose

Made with GAMMA

Structural Abstractions: Model vs. System Model vs. View

Model

Abstraction describing a subset of a system while ignoring irrelevant details

System Model

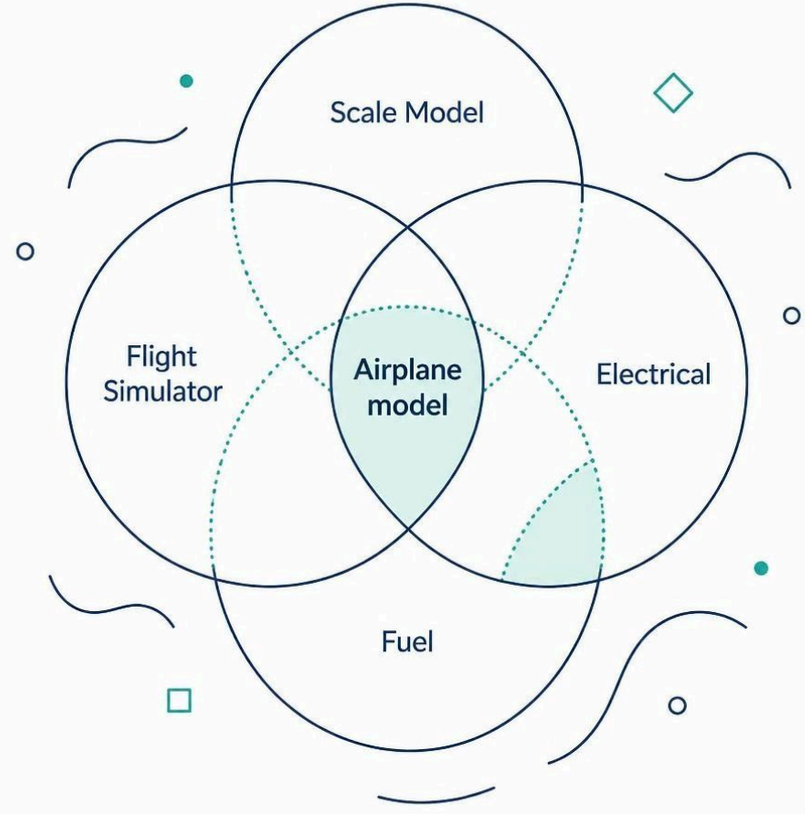
Complete set of all models created during development

View

Depiction focusing on a selected subset of a model for comprehensibility

Overlapping Views

Distinct views overlap when sharing underlying system components



Views depict specific subsets of a model; distinct views (like electrical and fuel) overlap where shared components connect. *Source: Textbook Ch. 2.3.1*

Made with GAMMA

Notations, Semantics & Industry Standards

Core Concepts

Notation

Graphical or textual rules for representing system views

Semantics

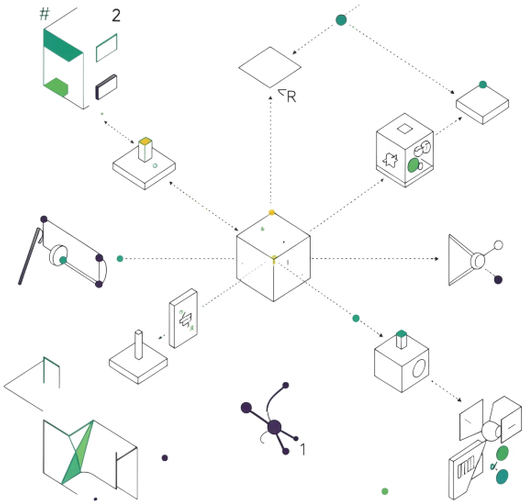
Well-defined meaning associated with notation symbols

Standardization

UML integrates precursor notations (OMT, Booch, OOSE) into an OMG standard

Industry Benchmark

Standardized semantics eliminate dialect misunderstandings across teams



UML unified precursor notations (OMT, Booch, OOSE) to create a single international standard with well-defined semantics. *Source: Textbook Ch. 2.1 & 2.3.1*

Made with GAMMA

Data Types, Abstract Data Types (ADTs) & OO Classes

Data Type

Programming language abstraction with a name, value set, and valid operations

Type Checking:
Compiler/runtime enforcement of operation validity (e.g., Java `int`)

Abstract Data Type (ADT)

Implementation-independent specification of value sets (e.g., `Set`)

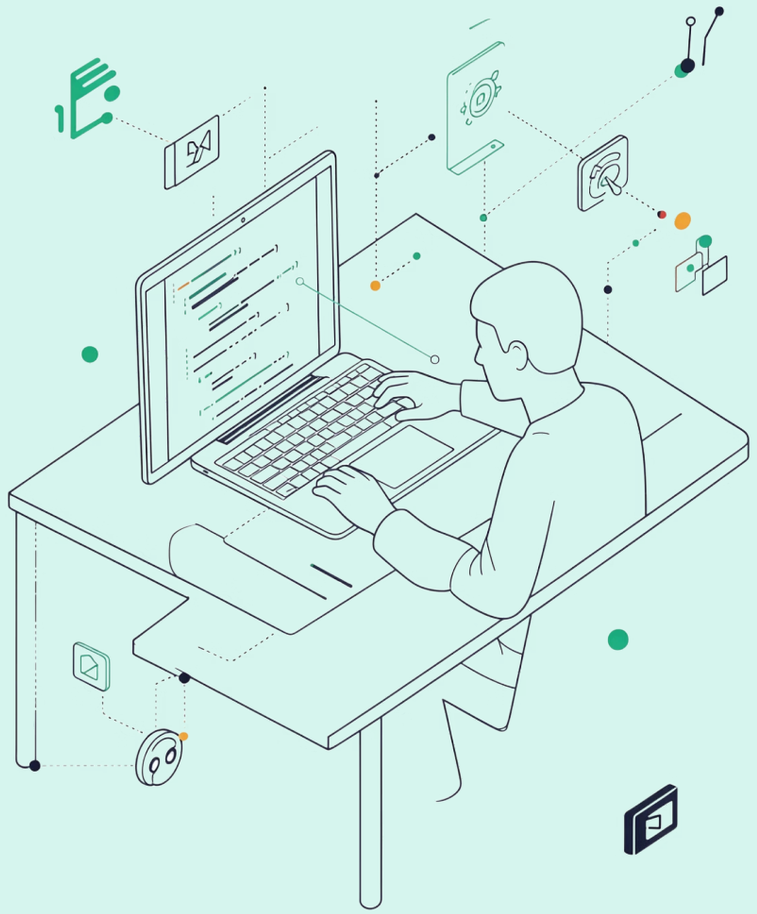
Specifies operations independently of storage representation

Encapsulation

ADTs hide internal storage decisions from the rest of the software system

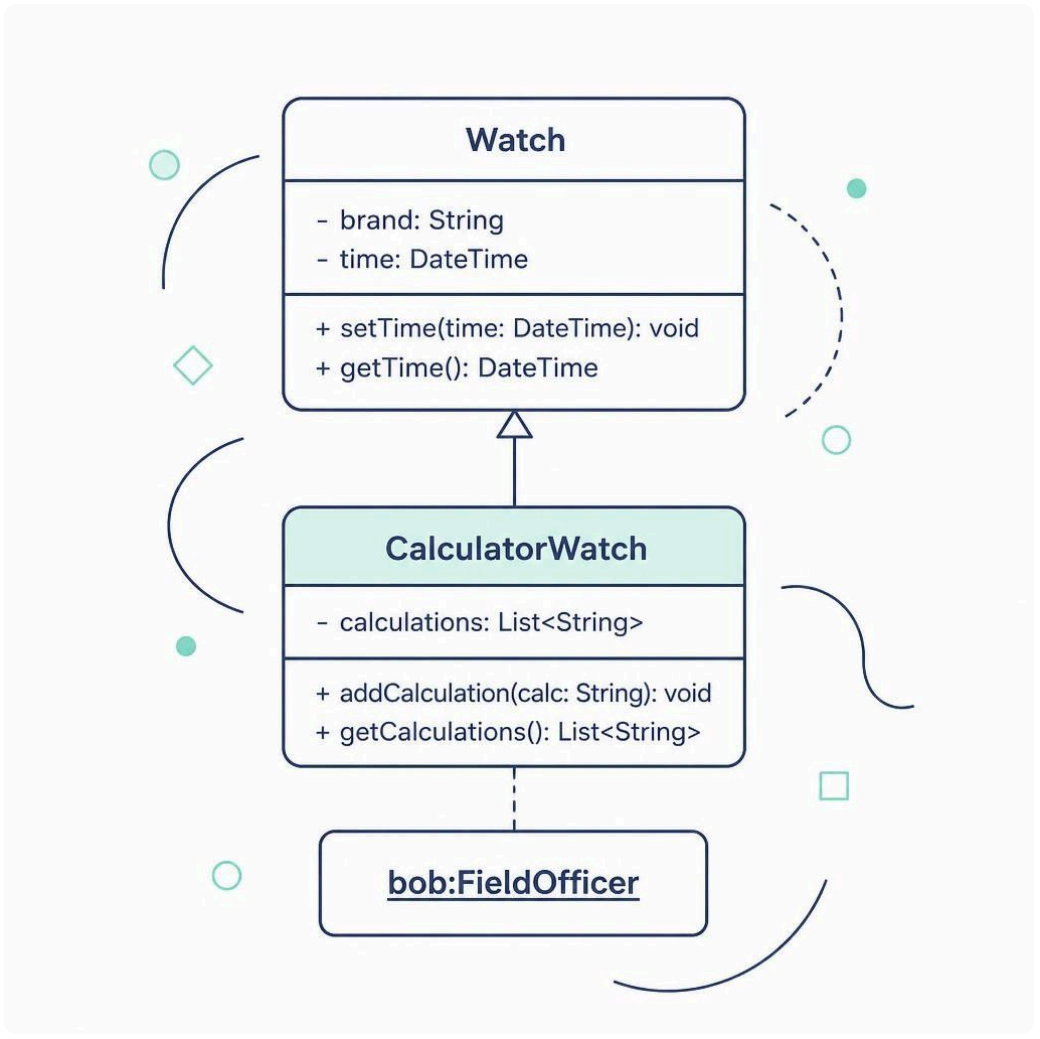
OO classes build on ADTs by adding inheritance

Source: Textbook Ch. 2.3.2 & 2.3.3



Made with GAMMA

Object-Oriented Abstraction: Classes, Abstract Classes & Objects



1

Class

Abstraction specifying shared **attributes** (structure) and **operations** (behavior)

2

Abstract Class

Superclass that cannot be directly instantiated — denoted by *italicized name*

3

Object (Instance)

Entity with individual state and identity — denoted by underlined name: `bob:FieldOfficer`

4

Inheritance

Specialization relationship where subclasses inherit attributes and operations

Instance names are always underlined in UML notation. *Source: Textbook Ch. 2.3.3*

Made with GAMMA

Dynamic Abstractions: Event Classes, Events & Messages

Event Class

Abstraction describing occurrences sharing a common system response

Event Instance

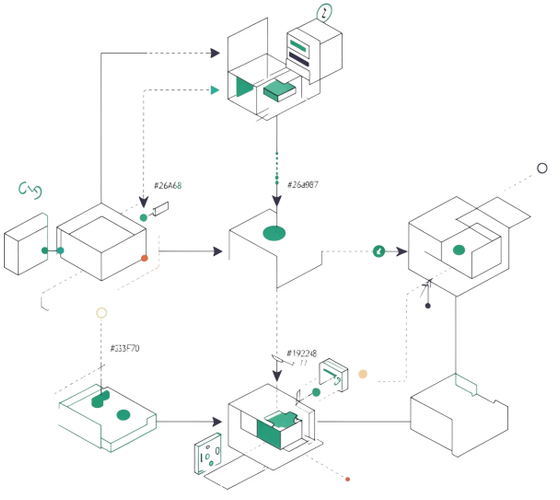
Concrete occurrence happening at a specific point in time (e.g., button press)

Message

Communication where a sender object requests operation execution from a receiver

Parameter Binding

Passed message arguments bind to operation parameters in the receiving object



Events represent instantaneous occurrences; messages request operation execution between objects and bind arguments to parameters. *Source: Textbook Ch. 2.3.4*

Made with GAMMA

The Three UML System Models: Functional, Object & Dynamic

Functional Model

Captures external system behavior from user perspective

Artifact: Use Case Diagrams

Object (Structural) Model

Captures static structure, classes, attributes, and associations

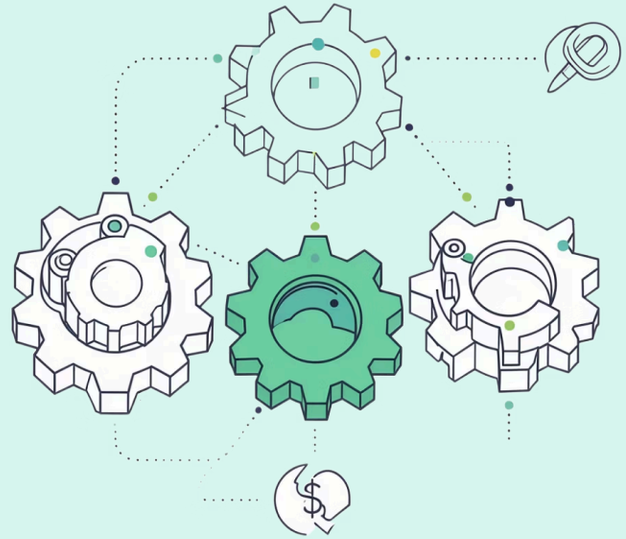
Artifact: Class Diagrams

Dynamic (Behavioral) Model

Captures internal interactions, state changes, and flows

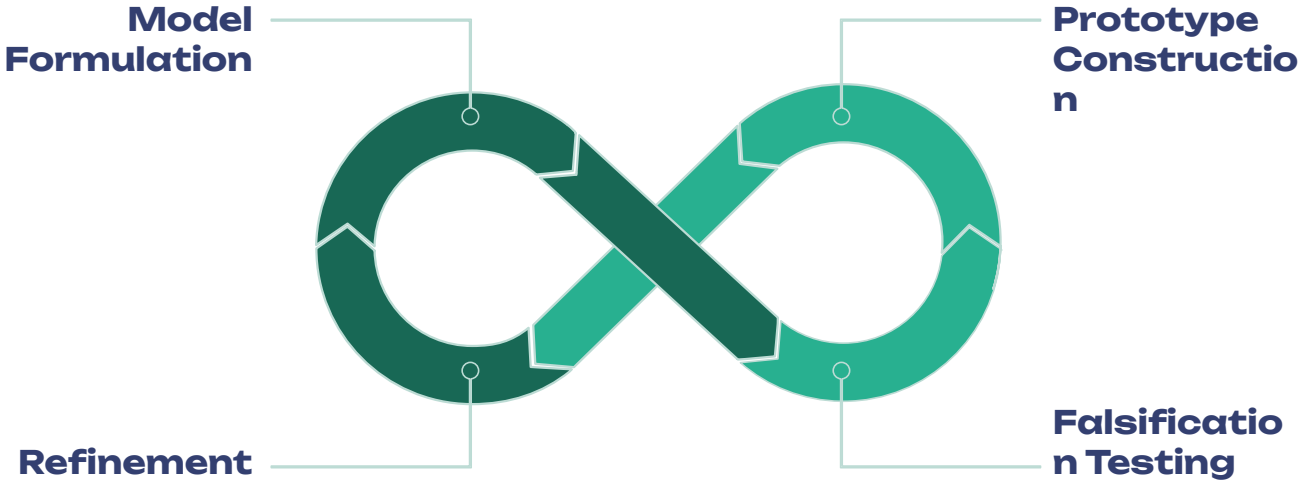
Artifacts: Sequence, State, Activity Diagrams

Tripartite Unity: Combined models form the complete software system architecture. *Source: Textbook Ch. 2.1*



Made with GAMMA

Elicitation & Validation: Falsification & Prototyping



Abstraction Tradeoff

Models simplify reality; relevant details included must be accurate

Falsification

Process of demonstrating that a model incorrectly represents reality

Prototyping

Building early experimental implementations to discover model flaws

Popper's Principle

Scientific validation relies on active attempts to falsify model assumptions

Karl Popper established that validation requires attempting to falsify assumptions; architects build prototypes to test model flaws early. *Source: Textbook Ch. 2.3.6*

Made with GAMMA

Application Domain Analysis vs. Solution Domain Design

Application Domain

Real-world problem space and user workflows

- `Aircraft`
- `TrafficController`

OO analysis models the problem space — what the system must do

Solution Domain

Software architecture space

- `MapDisplay`
- `FlightPlanDatabase`

OO design introduces solution objects like databases and UI controllers

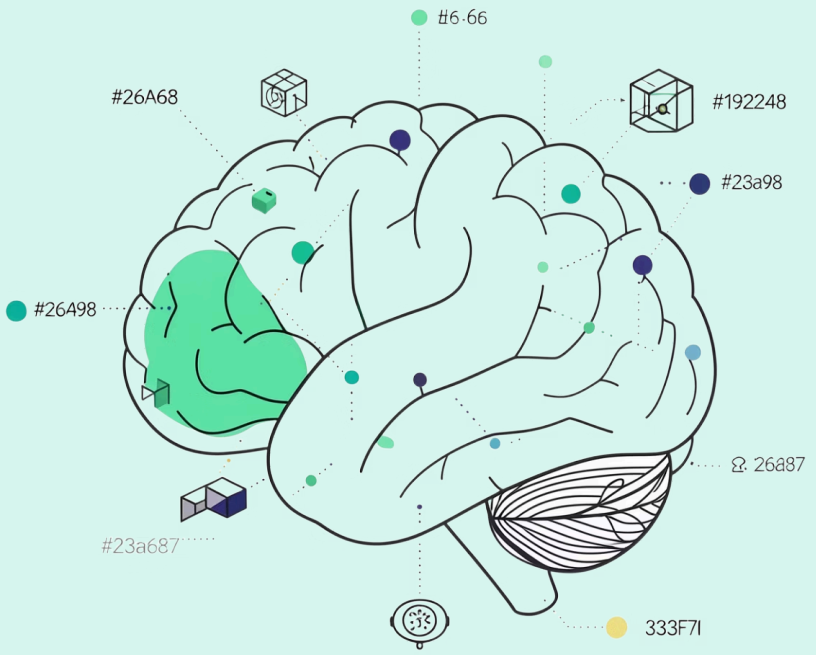
---

Key Concerns

- **Traceability:** Single UML notation provides seamless mapping from requirements to code
- **Risk:** Prematurely introducing solution-space objects into initial requirements analysis

Source: Textbook Ch. 2.3.5

Made with GAMMA



Cognitive Capacity Constraints: Miller's 7 ± 2 Rule

7±2

Chunks in Memory

Human short-term memory processes 7 ± 2 chunks of information at once

1956

Miller's Law

Fundamental rule guiding maximum diagram element density in UML

5–9

Elements Per View

Architectural rule: limit individual diagram views to 5–9 primary elements

❌ Cluttered Diagram

An 18-class diagram overwhelms short-term memory, making it impossible to reason about structure at a glance

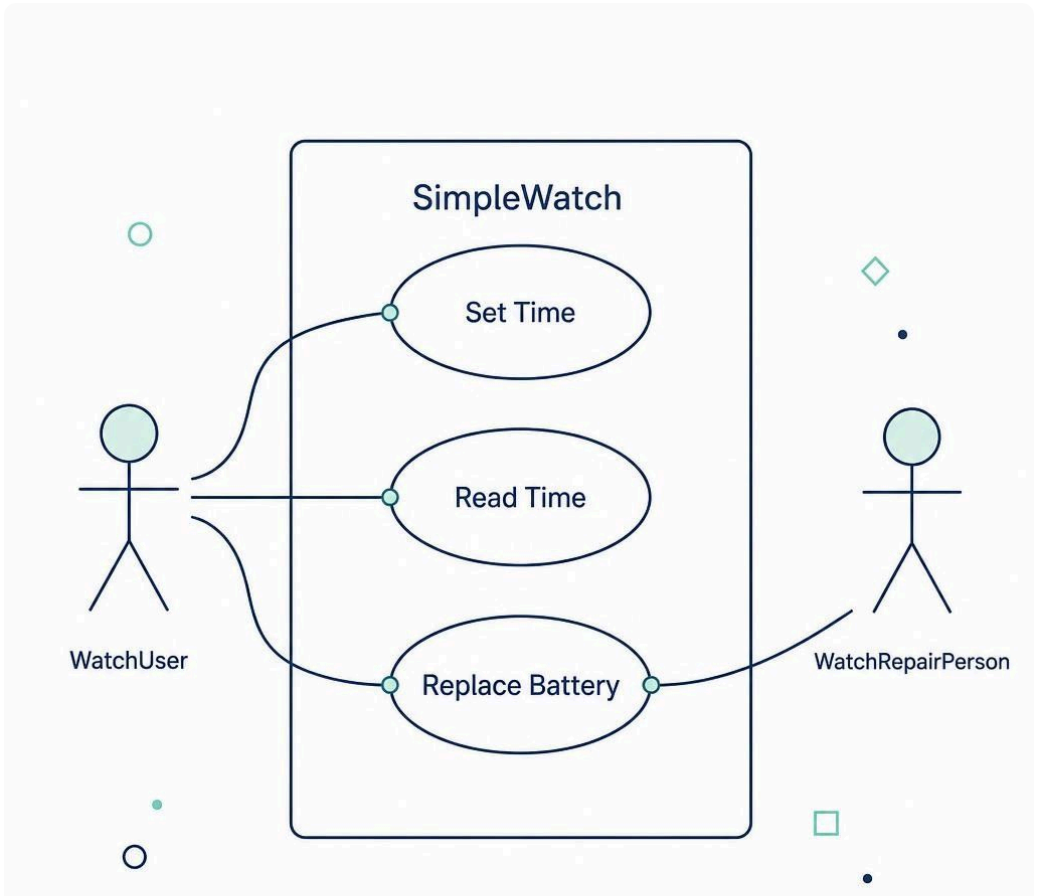
✅ Organized Packages

Decomposition: Group complex models into UML Packages to maintain clarity — a 3-package diagram stays within cognitive limits

Miller's law dictates that diagrams should contain no more than 7 ± 2 primary elements to remain comprehensible. *Source: Textbook Ch. 2.3.1 & 2.4.6*

Made with GAMMA

Functional View: Use Case Diagrams & System Boundaries



01

Functional Model Purpose

Describes system behavior from an external user's perspective

02

System Boundary

Graphical box surrounding use cases separating internal from external tasks

03

Actors

Placed **outside** the system boundary as stick figures or named boxes

04

Use Cases

Placed **inside** the system boundary as labeled ovals

System boundaries separate external actor roles from internal system use cases; actors reside outside the box. *Source: Textbook Ch. 2.2.1 & 2.4.1*

Made with GAMMA

Actors, Use Cases & External System Behavior

TEXTBOOK CH. 2.4.1

Core Concepts

Actor

External entity interacting with the system (user role, hardware device, external system)

Use Case

System function yielding a visible, measurable result for an actor

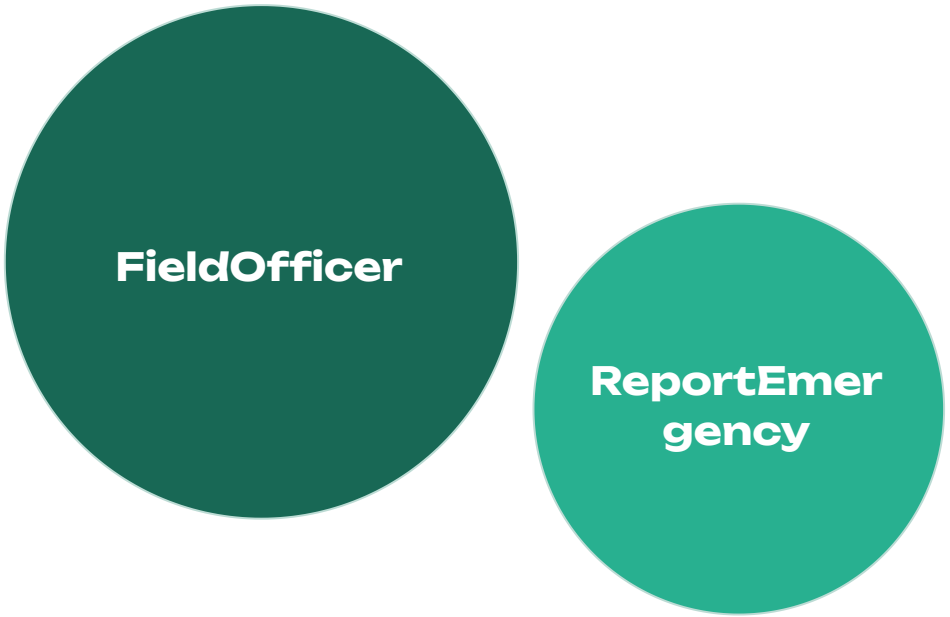
External Behavior

System behavior described strictly as perceived from actor viewpoints

Communication Relationship

Solid line denoting bidirectional information exchange or access

Diagram: FieldOfficer → ReportEmergency



Actors represent external roles; use cases yield measurable value to actors; solid lines denote communication access.

Made with GAMMA

Textual Use Case Specifications: Structured Templates

TEXTBOOK CH. 2.4.1

Diagrams are augmented with **6-field textual templates** adapted from Constantine & Lockwood. The flow of events uses 2 columns to separate actor and system steps.

Field	Description	Example: ReportEmergency
Name	Unique identifier for the use case	ReportEmergency
Participating Actors	All actors who interact with this use case	FieldOfficer, Dispatcher
Entry Conditions	Preconditions that must hold before execution begins	FieldOfficer is logged in and on duty
Exit Conditions	Postconditions that hold after successful completion	Emergency report is filed and acknowledged
Flow of Events	Numbered 2-column steps: Actor Action | System Response	1. Officer submits form → System validates and logs incident
Quality Requirements	Non-functional constraints (e.g., performance, security)	System must respond within **30 seconds**

Made with GAMMA

Functional Relationships: Communication & «include»

TEXTBOOK CH. 2.4.1

Relationship Types

Communication Line

Solid line representing actor interaction with a use case

«include» Relationship

Factors out common behavior shared across multiple use cases

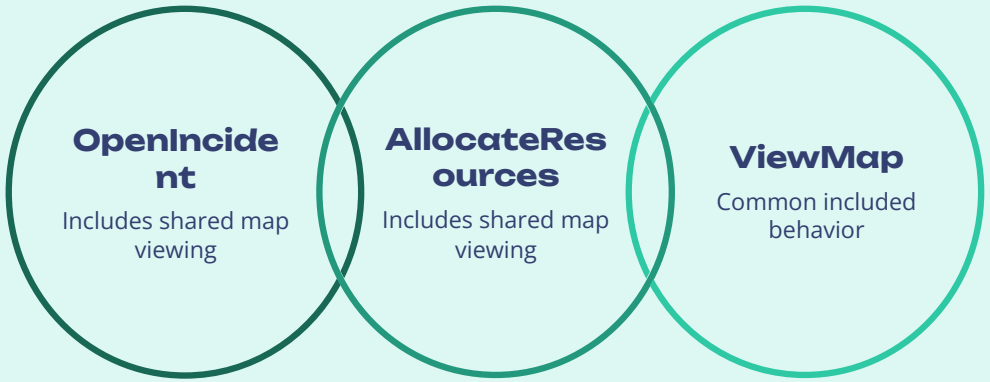
Arrow Syntax

Dashed open arrow pointing **from** including use case **to** included use case

Execution

Included behavior executes **unconditionally** as part of the main flow

Diagram: «include» → ViewMap



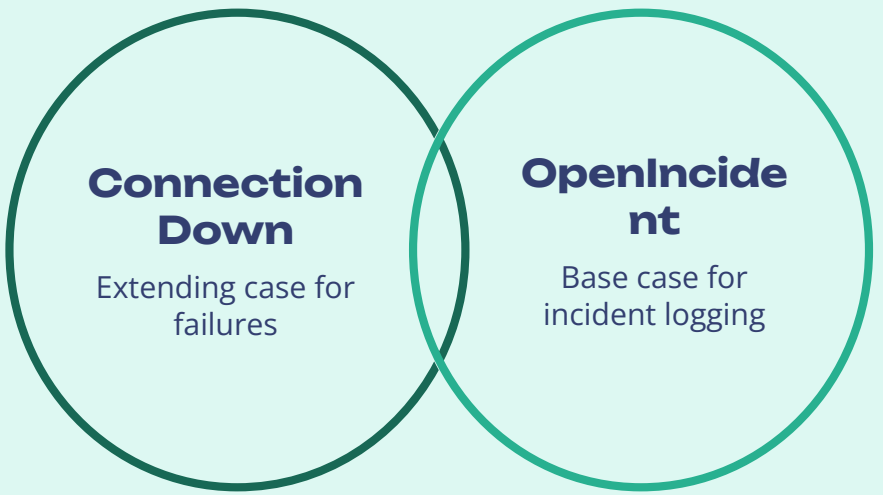
«include» factors out shared behavior; dashed arrow points from including case to the included case (ViewMap).

Made with GAMMA

Functional Relationships: «extend» for Exceptional Behavior

TEXTBOOK CH. 2.4.1

Diagram: «extend» → OpenIncident



«extend» isolates exception handling; dashed arrow points from extending case (ConnectionDown) back to base case.

Key Properties

«extend» Relationship

Models optional or exceptional behavior extending a base use case

Arrow Syntax

Dashed open arrow pointing **from** extending use case **to** base use case

Decoupling

Base use case remains **completely unaware** of extending use cases

Trigger

Extension flow executes **only** when specific extension conditions are met

Made with GAMMA

Functional Relationships: Use Case Inheritance vs. Generalization

TEXTBOOK CH. 2.4.1

Inheritance Properties

Use Case Inheritance

Specialized child use case refines a general super-use case

Notation Syntax

Solid line with a **hollow triangle** pointing to the super-use case

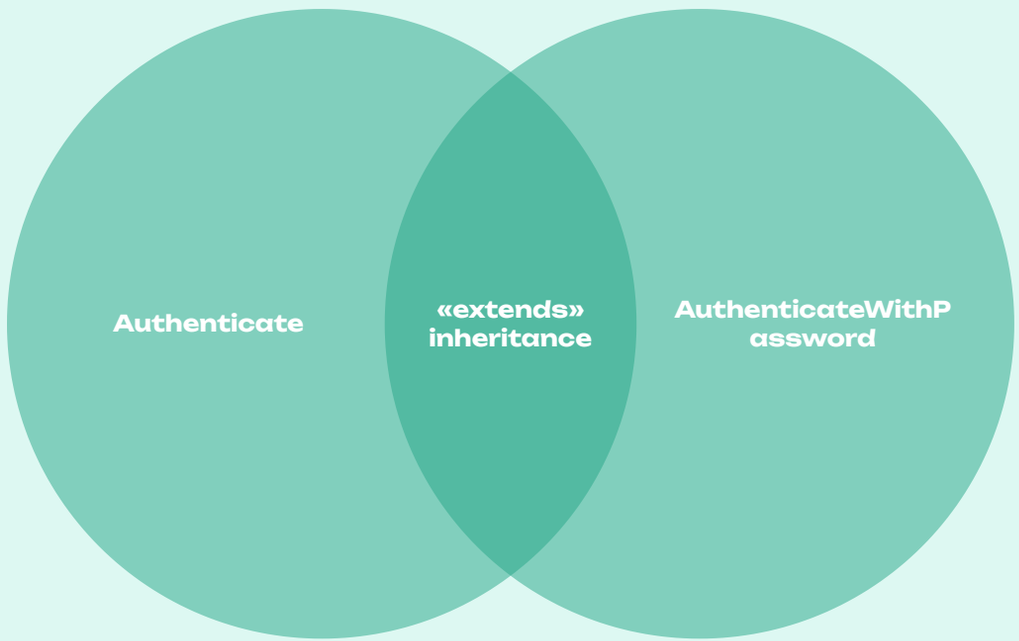
Inherited Elements

Child inherits participating actors, entry conditions, and exit conditions

Key Difference

Inheritance refines the same core task; **«extend»** adds an exceptional separate flow

Diagram: AuthenticateWithPassword → Authenticate



Inheritance specializes the same core task; hollow triangular arrow points toward the general super-use case.

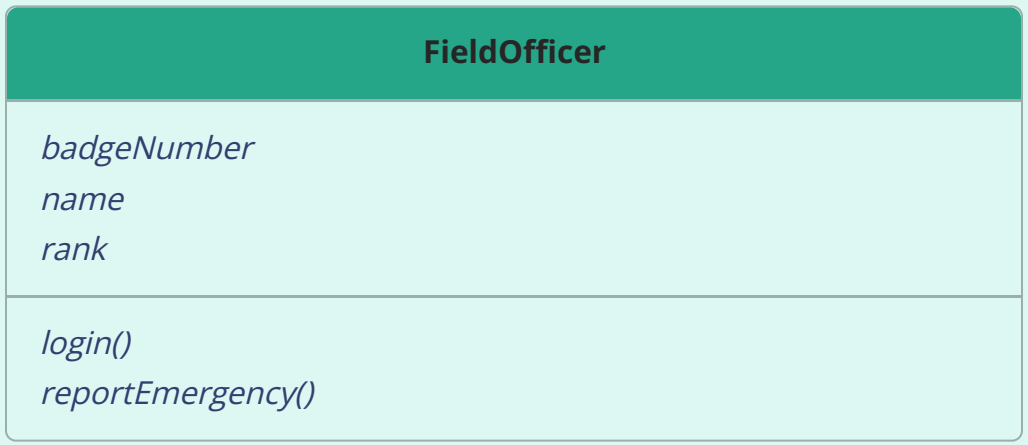
Made with GAMMA

Structural View: Class Diagrams & Object Instances

TEXTBOOK CH. 2.4.2

Class diagrams model **static system structure** — classes, attributes, operations, and associations. Object instance boxes represent concrete runtime snapshots.

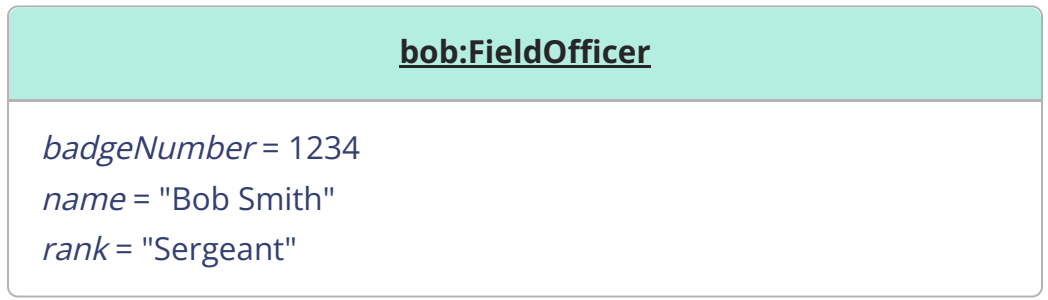
3-Compartment Class Box



Top = Class Name · Middle = Attributes (name:Type) · Bottom = Operations

Attribute data types are **omitted during analysis** and added during object design.

Object Instance Box



Underlined instance name (bob:FieldOfficer) with concrete attribute values assigned

Analysis Abstraction

Data types omitted at analysis phase; added during object design phase

Made with GAMMA

Association Constructs: Links, Multiplicity, Roles & Navigability

TEXTBOOK CH. 2.4.2

Construct Definitions

Link vs. Association

Link connects individual objects; **Association** connects classes

Role Labels

Text string on association ends defining the role of participating classes

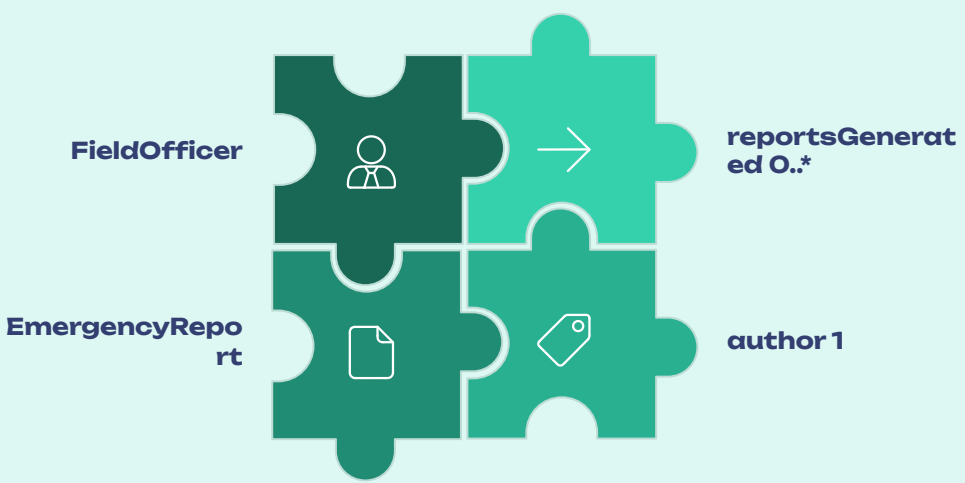
Multiplicity Bounds

Set of integers specifying valid object links: `1`, `0..*`, `1..*`

Navigability

Open arrowheads indicate directional reference traversal in memory

Diagram: FieldOfficer ↔ EmergencyReport



Multiplicities dictate valid link bounds; roles clarify association ends; navigability arrows indicate memory traversal.

Made with GAMMA

Advanced Associations: Aggregation, Association Classes & Qualifiers

TEXTBOOK CH. 2.4.2

Construct Definitions

Aggregation

Container-part relationship drawn with an **open diamond** at aggregate end

Association Class

Holds attributes/operations belonging to an association (dashed connection)

Qualified Association

Key attribute box (qualifier) that reduces association multiplicity

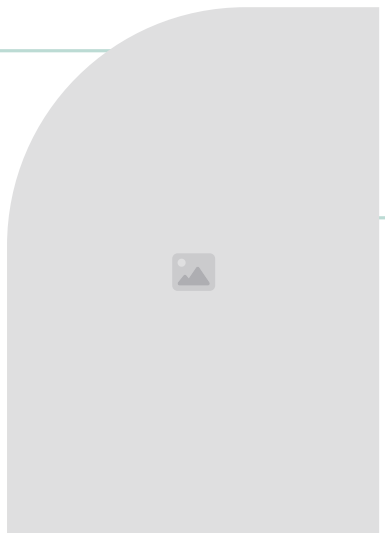
Efficiency

Qualifiers reduce **1-to-many** lookup multiplicities down to **1-to-1** indexed matches

Diagram: Association Class & Qualified Association

Association Class

Allocation shown linked to association via dashed line.



Qualified Association

Directory uses qualifier 'filename' to return single File.

Aggregation uses open diamonds; association classes hold relationship state; qualifiers reduce lookup multiplicity.

Made with GAMMA

Dynamic Behavioral View: Interaction & Sequence Diagrams

TEXTBOOK CH. 2.2.3 & 2.4.3 · PLANTUML SEQUENCE GUIDE

Core Elements

Interaction Purpose

Formalizes dynamic interaction and message exchange **chronologically**

Lifelines

Vertical dashed lines representing object existence over time

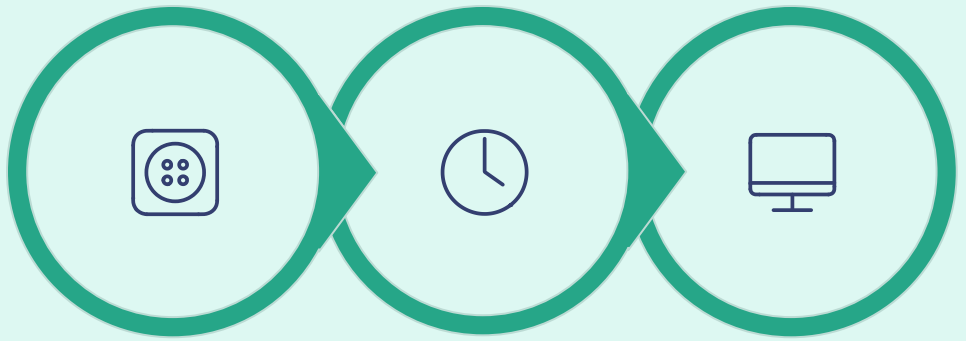
Stimuli / Messages

Horizontal solid arrows labeled with operation calls and parameters

Interaction Frames

Frame	Purpose
alt	Conditional branching between alternative flows
loop	Iteration over a repeated sequence of messages
ref	Reference to a sub-diagram defined elsewhere

Diagram: SetTime Sequence



pressButton

incrementHour

updateDisplay

Sequence diagrams show chronological message passing along vertical lifelines; interaction frames handle branching and loops.

Made with GAMMA

Object State Modeling: State Machine Diagrams & Transitions

TEXTBOOK CH. 2.2.4 & 2.4.4

Core Concepts

State Machine Purpose

Describes state sequences and event responses for a **single nontrivial object**

State

Condition satisfied by object attribute values (e.g., `Active`, `Closed`)

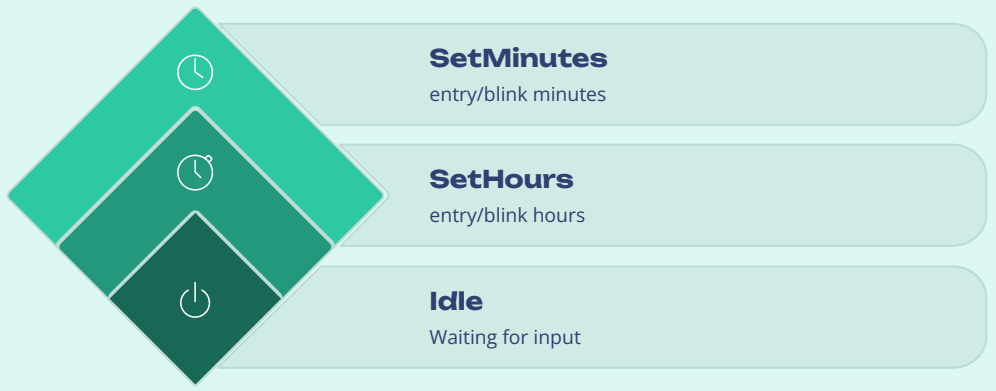
Transition

Triggered by external events; moves object from source state to target state

State Actions

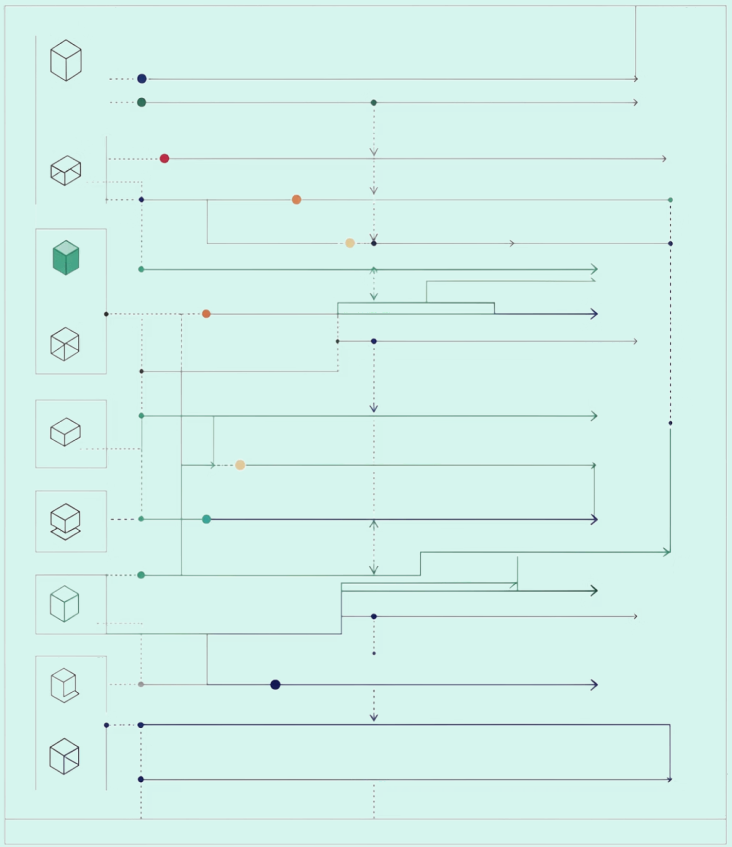
Action Type	Description
`entry/`	Executed when object enters the state (e.g., `entry/blink hours`)
`exit/`	Executed when object leaves the state
Transition Action	Executed during the transition itself (e.g., `pressBothButtons/beep`)

Diagram: SetTime State Machine



State machine diagrams focus on a single object; transitions trigger on events and can execute `entry/` and `exit/` actions.

Made with GAMMA



Workflow Modeling: Activity Diagrams, Swimlanes & Concurrency

Activity Diagram Purpose

Flowchart-style diagram representing operational workflows and data flows

Nodes & Edges

Rounded rectangles (activities), directed arrows (control flow)

Concurrency

Fork bars split execution into parallel threads; join bars synchronize threads

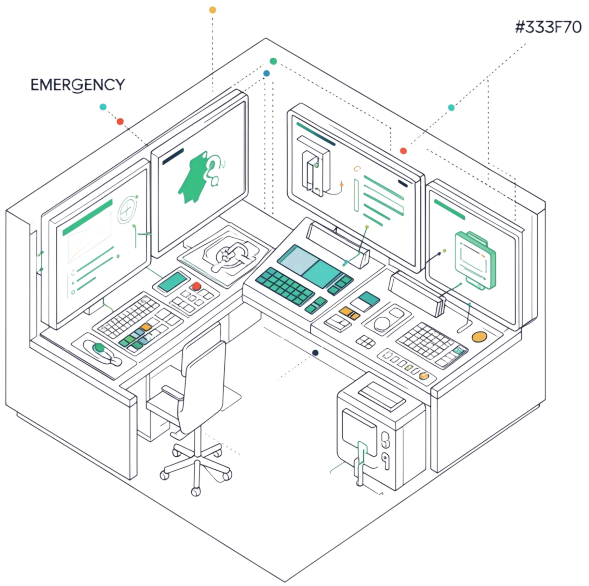
Swimlanes (Partitions)

Group activities vertically by responsible organizational role or object

Activity diagrams depict workflow flows; thick bars represent concurrency forks/joins; swimlanes organize responsibilities. Source: Textbook Ch. 2.2.5 & 2.4.5.

Made with GAMMA

Case Study Introduction: FRIEND System Context & Stakeholders



FRIEND

First Responder Interactive Emergency Navigational Database

Domain Space: Real-time emergency incident management and dispatch assistance

Objective: Coordinate emergency reporting, incident creation, and unit allocation wirelessly

---

Key Actors

- `FieldOfficer` — mobile field user
- `Dispatcher` — central command user

FRIEND connects wireless field terminals used by police/firefighters with central dispatchers. Source: Textbook Ch. 2.4.1.

Made with GAMMA

FRIEND Functional Model: Use Case Diagram & System Boundary

System Boundary

FRIEND Emergency Management System

Actors

- `FieldOfficer` (Left)
- `Dispatcher` (Right)

Communication

Solid lines denote bidirectional access and messaging permissions

Made with GAMMA

FRIEND Use Case Specification: ReportEmergency Template Walkthrough

Use Case

ReportEmergency

Actors

Initiated by FieldOfficer, communicates with Dispatcher

Quality Constraint

System must deliver notification and acknowledgment within **30 seconds**

Step	Actor Action	System Response
1	FieldOfficer submits emergency location and level	System receives and validates submission
2	—	System notifies Dispatcher of emergency
3	Dispatcher acknowledges the emergency	System confirms acknowledgment to FieldOfficer

The textual specification structures the interaction; the main flow alternates between actor inputs and system responses. Source: Textbook Ch. 2.4.1.

Made with GAMMA

FRIEND Exception Handling: ConnectionDown «extend» Relationship

Base Use Cases

- `OpenIncident`
- `AllocateResources`

Extending Use Case

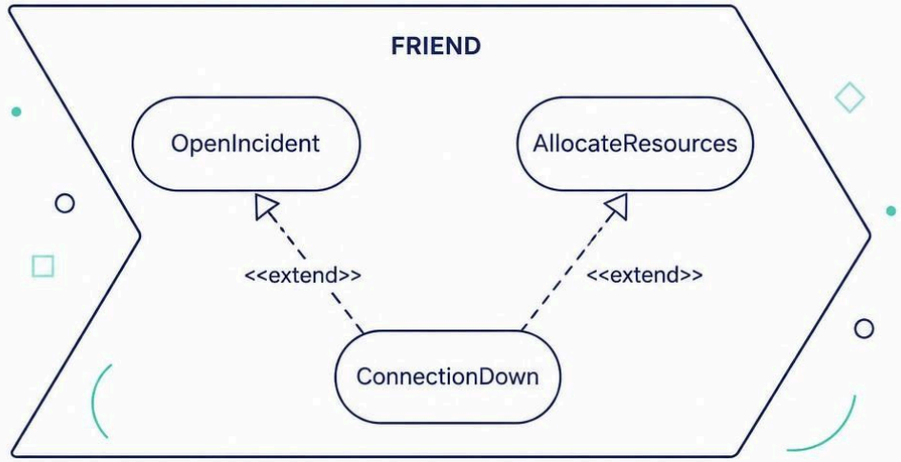
`ConnectionDown`

Trigger Condition

System detects lost wireless connection between `FieldOfficer` and `Dispatcher`

Decoupling

Isolates network fault handling from standard incident workflows



Made with GAMMA

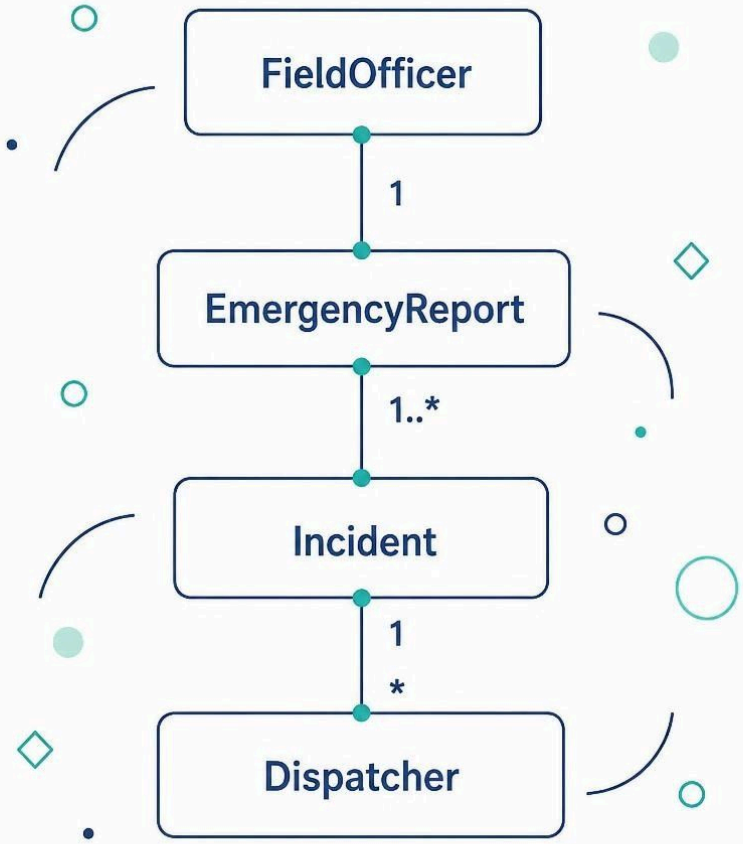
FRIEND Static Domain Model: Emergency Incident Class Diagram

Domain Classes

Domain Rule

An Incident requires at least 1 (1..*) emergency report to be created

Static domain model captures core entities; multiplicities enforce domain rules. Source: Textbook Ch. 2.4.2.



FieldOfficer

name:String
badgeNumber:Integer

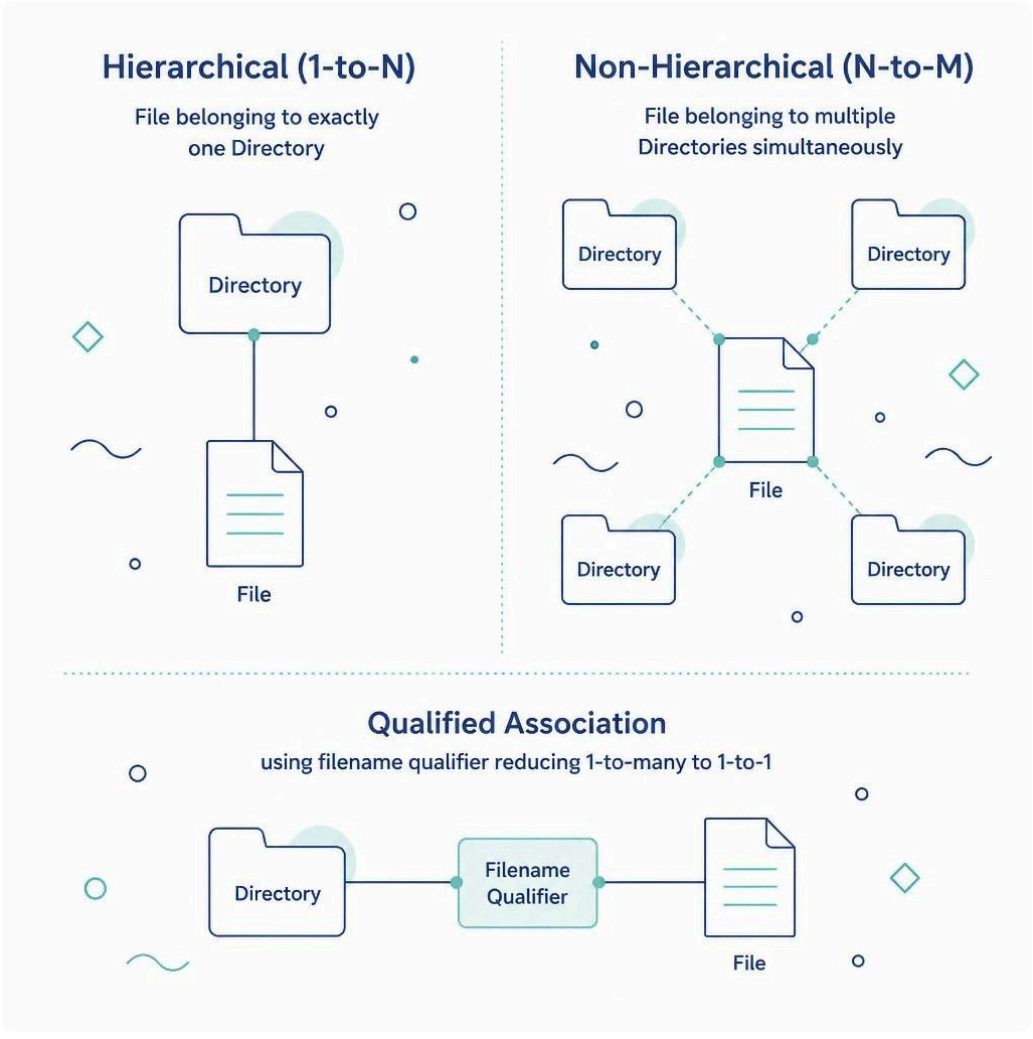
EmergencyReport

Incident

Dispatcher

Made with GAMMA

Structural Complexity Tradeoffs: Hierarchical vs. Non-Hierarchical File Systems



Hierarchical (1-to-N)

File belongs to exactly 1 directory; simple creation/deletion semantics

Non-Hierarchical (N-to-M)

File belongs to multiple directories simultaneously; flexible but complex

Qualified Association

Uses `[filename]` qualifier to reduce 1-to-many lookups to 1-to-1 matches

System Scope Impact

N-to-M associations force addition of complex link, unlink, and deletion use cases

> Multiplicity choices drive functional scope — changing from 1-to-N to N-to-M forces new use cases for link/unlink operations.

Made with GAMMA

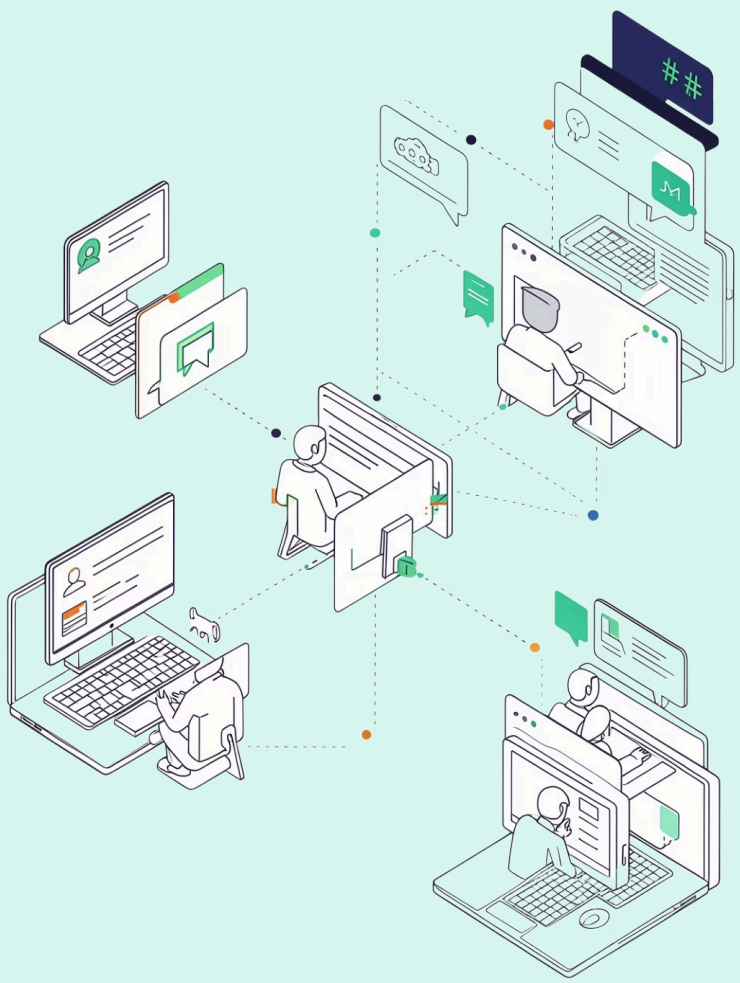
FRIEND Dynamic Behavior: Sequence Trace for warehouseOnFire Scenario

Participating Objects

- bob:FieldOfficer
- alice:FieldOfficer
- john:Dispatcher
- FRIEND



Sequence traces formalize dynamic interaction; walking scenario messages reveals missing class operations. Source: Textbook Ch. 2.4.1 & 2.4.3.



Made with GAMMA

FRIEND Implementation: PlantUML Text-to-Diagram Syntax & Execution

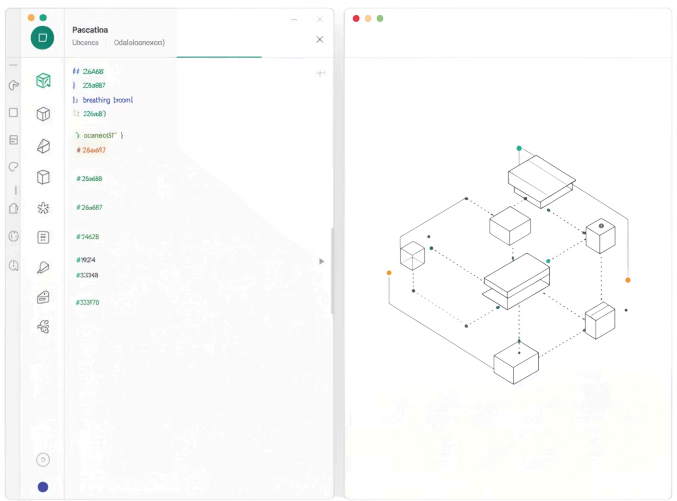
Text-to-Diagram

Plain text scripts serve as single source of truth; layouts auto-rendered

---

Sequence Syntax

```
Alice -> John: ReportEmergency(location, level)
```



Activation Syntax

```
activate John
deactivate John
```

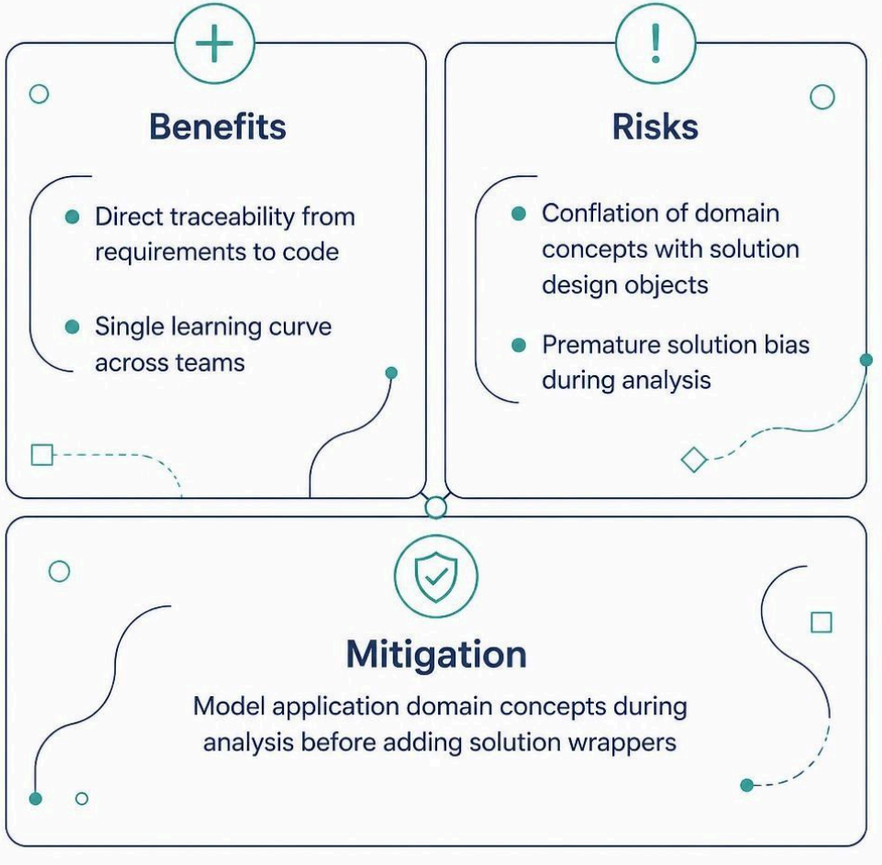
Tooling Integration

- Java CLI: `java -jar plantuml.jar`
- VS Code extensions
- Docker

PlantUML converts plain text scripts into clean UML diagrams, integrating easily into Git and IDE workflows. Source: Quick Start Guide to PlantUML; Sequence diagram - PlantUML.

Made with GAMMA

Tradeoff Analysis: Single Unified Notation Strengths vs. Risks



Strengths

- Direct traceability from requirements to code
- Single learning curve across teams

Risks

- Developers can easily conflate real-world domain concepts with solution design objects

Mitigation

Model application domain concepts during analysis before adding solution wrappers

Single unified notation enables direct traceability but risks premature solution bias during analysis. Source: Textbook Ch. 2.3.5; Design Brief.

Made with GAMMA

Multiplicity Tradeoffs: Associations & System Functional Scope

SLIDE 40 | TEXTBOOK CH. 2.4.2

Association multiplicities are not merely notational choices — they directly dictate backend database schemas and the full set of functional use cases a system must support. Selecting multiplicities based on verified domain rules rather than theoretical flexibility is a foundational discipline of rigorous analysis modeling.

Hierarchical (1-to-N)

- Low structural complexity
- Simple reference counting semantics
- Restricted flexibility in domain representation
- Fewer cascading side effects on deletion

Non-Hierarchical (N-to-M)

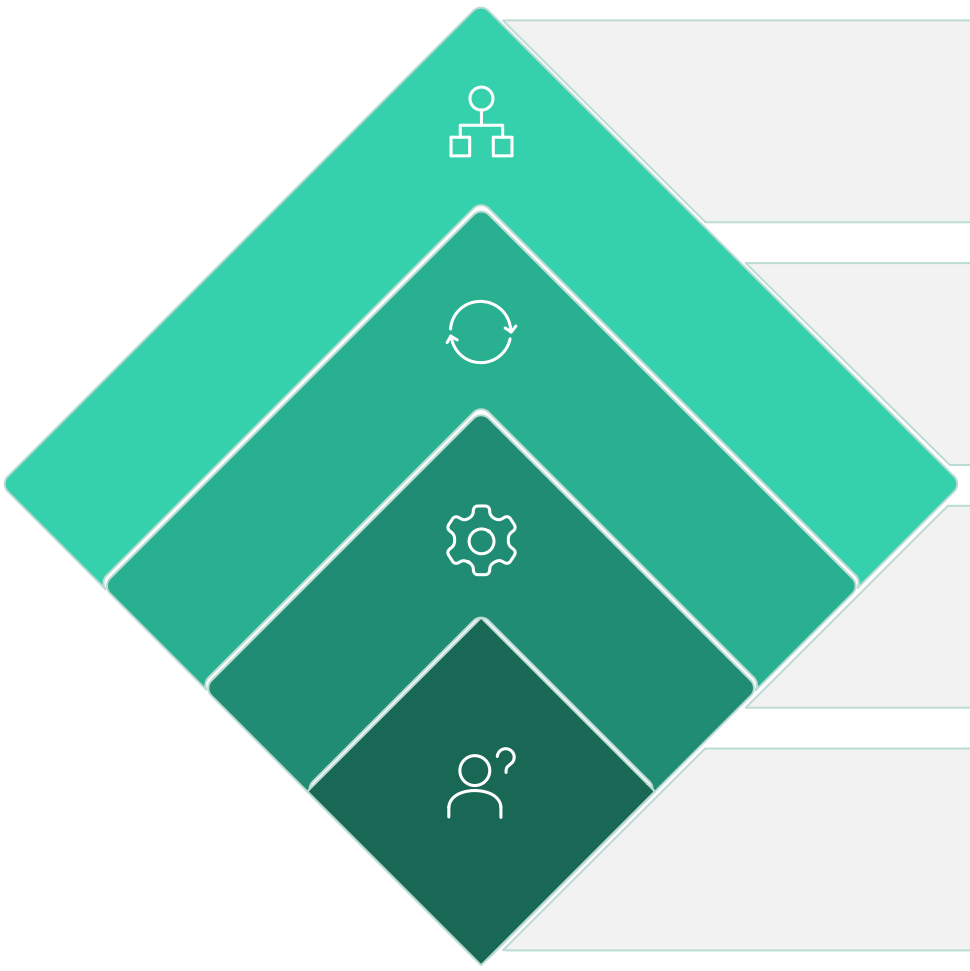
- High flexibility in domain representation
- Complex cascading deletion semantics
- Higher implementation and maintenance cost
- Requires junction/association classes in relational schemas

Decision Rule

Select multiplicities based on **verified domain rules** rather than theoretical flexibility. Each multiplicity bound expands or contracts the required set of use cases the system must implement and test.

Architectural Impact

A change from `1` to `0..*` on an association end is not cosmetic — it introduces new creation, deletion, and navigation use cases that must be fully specified and implemented.



Derive Required Use Cases

Map to functional scope

Return to Domain Analysis

Revisit requirements

Apply Exact Multiplicity Bound

Set precise multiplicity

Domain Rule Verified?

Check association rules

Made with GAMMA

Common Modeling Pitfalls: Over-Specifying Design in Analysis Models

SLIDE 41 | TEXTBOOK CH. 2.4.1 & 2.4.2

Analysis models must remain technology-agnostic and behavior-focused. Premature design decisions embedded in analysis artifacts corrupt the abstraction layer and propagate implementation bias into requirements. The four pitfalls below represent the most common violations observed in practice.

❌ Incorrect Practice

Pitfall 1: Premature Typing

Specifying language-level data types such as `java.util.Vector` during initial analysis. Analysis attributes should use domain-level types only.

Pitfall 2: Procedural Decomposition

Drawing class diagrams as procedural flowcharts rather than state/behavior encapsulations. Classes must model objects, not functions.

Pitfall 3: Monolithic Use Cases

Writing monolithic inline flows instead of factoring out shared sub-flows with «include» and exceptional flows with «extend».

Pitfall 4: Missing System Boundary

Omitting system boundary rectangles, leaving actor scope ambiguous and the system's responsibility undefined.

✅ Correct Practice

Use Domain-Level Types

Specify attributes as `String`, `Integer`, or domain concepts. Defer language-specific types to the design phase.

Encapsulate State & Behavior

Model classes as encapsulations of domain state and behavior. Sequence diagrams capture procedural flow separately.

Factor with «include» / «extend»

Extract mandatory shared sub-flows into «include» targets. Isolate exceptional flows as «extend» extensions.

Draw System Boundary Rectangle

Always enclose system use cases within a labeled rectangle. Actors outside the boundary; use cases inside.

Made with GAMMA

Best Practices: Organizing Complex Models with UML Packages

SLIDE 42 | TEXTBOOK CH. 2.4.6 · SYLLABUS CEN 4065C

As system models grow, individual diagrams exceed human cognitive capacity. UML packages provide a formal grouping construct that partitions large models into manageable sub-scopes, each representable as a self-contained diagram.

Package Fundamentals

- **Definition:** Grouping construct for organizing related model elements into named sub-scopes
- **Visual Notation:** Tabbed folder rectangle enclosing related classes or use cases
- **Architectural Value:** Deconstructs large system models to respect cognitive capacity limits
- **Cognitive Principle:** Miller's Law — keep diagrams within **7 ± 2 elements** per view

FRIEND System Package Structure

IncidentManagement

Use cases for reporting, dispatching, and tracking active incidents in real time.

IncidentArchive

Use cases for storing, retrieving, and auditing historical incident records.

SysAdministration

Use cases for user management, system configuration, and access control.

Packages are not just organizational conveniences — they define dependency boundaries. Cross-package dependencies should be minimized and made explicit via «import» or «access» relationships.

Made with GAMMA

Extension Mechanisms: Stereotypes & Constraints

SLIDE 43 | TEXTBOOK CH. 2.4.7

UML's built-in extension mechanisms allow modelers to classify architectural roles and enforce semantic business rules without departing from standard notation. Stereotypes and constraints are the two primary instruments for this purpose.

Stereotypes «...»

Extends UML meta-model elements to classify architectural roles. Applied to classes, components, and relationships.

«entity»

Holds **persistent domain data and state**. Corresponds to database-backed objects. Examples: `Incident`, `Report`, `User`.

«boundary»

Handles **interaction between external actors and the internal system**. Examples: `IncidentForm`, `LoginScreen`.

«control»

Coordinates **execution logic, sequencing, and business rules**. Examples: `IncidentController`, `AuthManager`.

Constraints {...}

Formal or informal semantic rules attached to model elements to enforce domain-specific invariants and ordering requirements.

Informal Constraint

Natural language rule enclosed in braces: `{ordered by time of receipt}` applied to an association end specifying collection ordering.

Formal Constraint (OCL)

Object Constraint Language expressions for machine-verifiable invariants: `{self.age >= 18}`.

Placement

Constraints may be placed adjacent to the constrained element, inside a note symbol, or on an association end.

Made with GAMMA

Active Learning Exercise 1: Distinguishing «include» vs. «extend»

SLIDE 44 | TEXTBOOK CH. 2.4.1 · EXERCISE 2-4

Scenario: Designing an Automated Teller Machine (ATM) software system. Two requirements must be classified using the correct UML use case relationship and arrow direction.

Requirement A

All monetary transactions require user PIN authentication.

Task

Identify the relationship type («include» or «extend») and specify the correct arrow direction between the base use case and the sub-use case.

Requirement B

If ATM cash drops below threshold during a withdrawal, the system alerts the bank supervisor.

Task

Identify the relationship type («include» or «extend») and specify the correct arrow direction between the base use case and the extension use case.

Solution: Requirement A → «include»

PIN authentication is **mandatory** for every monetary transaction. The base use case (WithdrawCash) unconditionally includes («include») the sub-use case (AuthenticatePIN). Arrow direction: base → included. The included use case is always executed.

Solution: Requirement B → «extend»

The supervisor alert is **exceptional** — triggered only when cash falls below threshold. The extension use case (AlertSupervisor) extends («extend») the base use case (WithdrawCash). Arrow direction: extension → base. The base use case is unaware of the extension.

Key Distinction: «include» = mandatory shared behavior, arrow from base to included. «extend» = conditional exceptional behavior, arrow from extension to base.

Made with GAMMA

Active Learning Exercise 2: Class Diagram & Multiplicity Elicitation

SLIDE 45 | TEXTBOOK CH. 2.4.2 · EXERCISE 2-7

Scenario: University course registration system with three domain classes: `Course`, `Section`, and `Student`. Domain rules have been elicited from stakeholders and must be translated into exact multiplicity bounds on association ends.

Domain Rule 1

A `Course` has **one or more** Sections.

Multiplicity on `Section` end: `1..*`

Domain Rule 2

A `Section` belongs to **exactly one** Course.

Multiplicity on `Course` end: `1`

Domain Rule 3

A `Student` enrolls in **zero to six** Sections.

Multiplicity on `Section` end: `0..6`

Domain Rule 4

A `Section` has **five to thirty** Students.

Multiplicity on `Student` end: `5..30`

Task

Sketch the static class diagram showing the three class boxes connected by association lines. Assign the exact multiplicity bounds derived above to each association end. Verify that each bound is traceable to a specific domain rule.

Association	Left End (Multiplicity)	Right End (Multiplicity)	Tracing Rule
Course — Section	`1`	`1..*`	Rules 1 & 2
Section — Student	`5..30`	`0..6`	Rules 3 & 4

Made with GAMMA

Active Learning Exercise 3: Sequence Diagram Message Flow & PlantUML Parsing

SLIDE 46 | TEXTBOOK CH. 2.4.3 FIG. 2-34 · PLANTUML SEQUENCE GUIDE

Scenario: Trace the sequence of messages when `WatchUser` presses buttons 1 and 2 simultaneously on the `2Bwatch` device. The interaction involves two internal lifelines and produces a visible display effect.

Message Flow Specification

01

Actor Initiates

`WatchUser` presses buttons 1 and 2 simultaneously. This is the initiating external actor event.

02

First Lifeline Activated

`:2BwatchInput` receives the message `pressButtons1And2()`. This is the first activated internal lifeline.

03

Second Lifeline Triggered

`:2BwatchInput` sends `blinkHours()` to `:2BwatchDisplay`, producing the visible output.

Task: Three Questions

Q1: Initiating Actor

Who initiates the sequence? →

Made with GAMMA

Comprehensive Summary: Synthesizing Functional, Structural & Behavioral Views

SLIDE 47 | TEXTBOOK CH. 2.1 · SYLLABUS CEN 4065C

A complete software architecture specification integrates three complementary model types. No single model is sufficient — each captures a distinct dimension of system behavior, and together they form the unified system specification required for implementation and verification.

Model Type	Primary UML Diagram	Key Constructs	Purpose
Functional Model	Use Case Diagram	Actors, Use Cases, System Boundary, «include», «extend»	Defines external system boundaries, actors, and user-visible use cases
Structural Model	Class Diagram	Classes, Attributes, Operations, Associations, Multiplicities, Stereotypes	Defines static architecture, classes, attributes, operations, and associations
Behavioral Model	Sequence / State / Activity Diagrams	Lifelines, Messages, States, Transitions, Guards, Actions	Formalizes dynamic interaction, state transitions, and operational workflows

1

Functional

What the system does from the user's perspective. Scope and actor boundaries.

2

Structural

How the system is organized. Static class architecture and relationships.

3

Behavioral

How the system behaves over time. Dynamic message flows and state changes.

4

Integrated Specification

All three models unified into a complete, verifiable system architecture.

Made with GAMMA

Architectural Takeaways: Intentional Modeling for System Engineering

SLIDE 48 TEXTBOOK CH. 2 SUMMARY · DESIGN BRIEF

Five core principles distilled from the full chapter. These are not stylistic preferences — they are engineering disciplines that directly affect model correctness, implementation fidelity, and system quality.

1 Intentional Abstraction

Filter out implementation clutter to highlight architectural drivers. Analysis models must remain technology-agnostic and focused on domain behavior, not platform specifics.

2 Respect Cognitive Limits

Keep diagrams within **7 ± 2 elements** using packages. Miller's Law applies directly to diagram readability — exceeding this threshold degrades comprehension and introduces modeling errors.

3 Decouple Exceptions

Use `«extend»` to keep base use case flows clean. Exceptional and conditional behaviors must be isolated from the primary success scenario to preserve base case clarity.

4 Analyze Multiplicity Early

Association bounds directly drive system functional scope. A multiplicity decision made in analysis propagates into database schema design, API contracts, and test case coverage.

5 Validate via Falsification

Build prototypes to test and falsify model assumptions early. Models are hypotheses — they must be challenged against real system behavior before implementation commitments are made.

Made with GAMMA

Practice Problem & Homework: Train System Ticket Distributor

SLIDE 49 | TEXTBOOK CH. 2 EXERCISES 2-4, 2-6, 2-10, 2-13 · PLANTUML QUICK START GUIDE

Homework Task: Construct a complete PlantUML model set for a Train Ticket Distributor system. The assignment spans all three model types covered in Chapter 2. Submit raw PlantUML `.puml` text scripts and generated `.png` diagram images.

1

Part A — Functional Model

Deliverable: Use Case Diagram

- `BuyOneWayTicket`
- `BuyWeeklyCard`
- `TimeOut`
- `DistributorOutOfChange`

Include system boundary rectangle, actors, and all applicable `«include»` / `«extend»` relationships.

2

Part B — Structural Model

Deliverable: Class Diagram

- `Ticket` — with attributes and operations
- `Tariff` — with attributes and operations
- `Payment` — with attributes and operations

Assign exact multiplicity bounds to all association ends. Use domain-level attribute types only.

3

Part C — Behavioral Model

Deliverable: Sequence Diagram

Trace the first **5 messages** for a complete ticket purchase interaction. Identify initiating actor, all activated lifelines, and message signatures.

Reference exercises: **2-4, 2-6, 2-10, 2-13**.

Submission Format: Raw PlantUML `.puml` script files AND rendered `.png` diagram images. Use the PlantUML CLI command: `java -jar plantuml.jar diagram.puml` to generate PNG output from script files.

Made with GAMMA